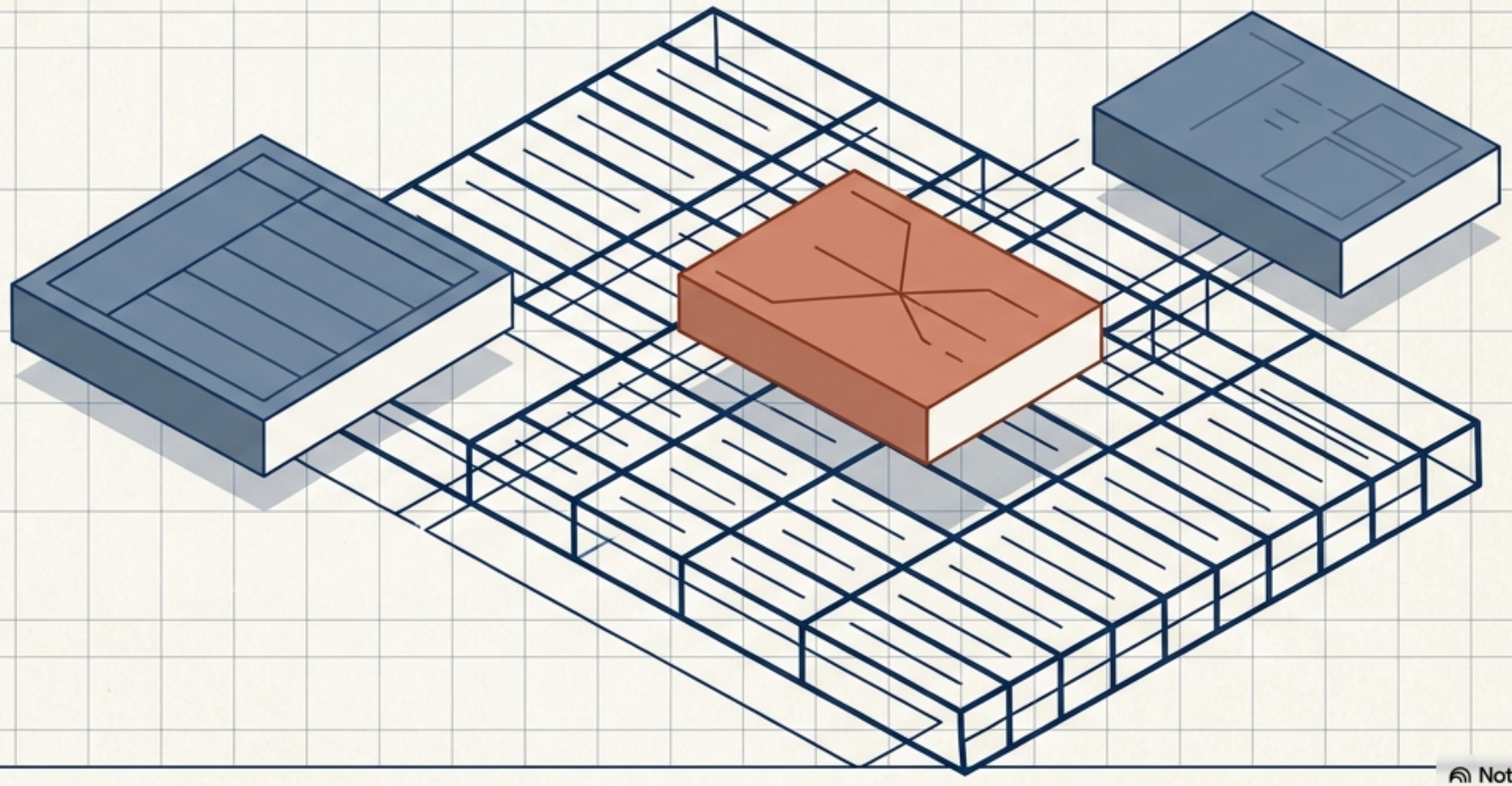
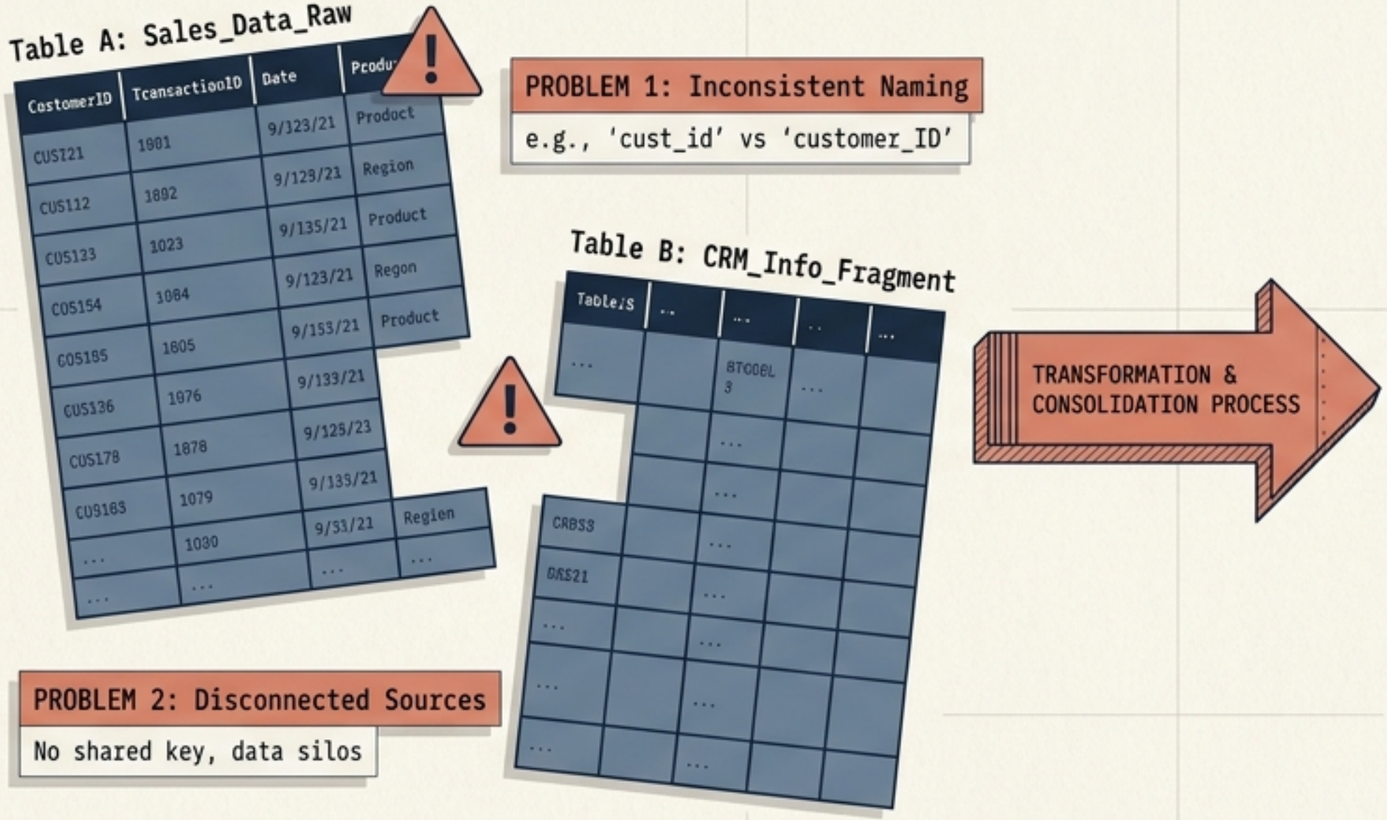
Data Architecture with Pandas

A Visual Blueprint for Renaming and Combining Datasets



THE CONSOLIDATION PROBLEM

Raw data rarely arrives ready for analysis. It comes from disparate sources with overlapping schemas, misnamed headers, and fragmented structures.



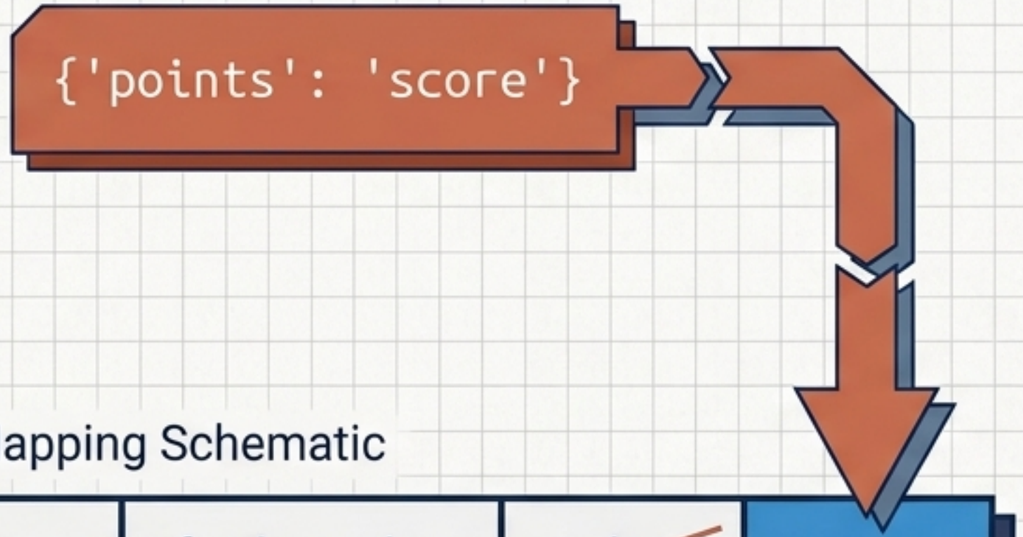
ARCHITECTED DATA TABLE: Unified_Sales_CRM_Model

CustomerID	TransactionID	Date	Product	Region	SalesAmount	LeadSeurce	Status
CUS135	22508	9/21/2023	Product	USA	$25.80	Creator	Complete
CUS126	22501	9/23/2022	Rogon	South	$25.80	LeadSource	Status
CUS127	25502	9/21/2822	Product	USA	$15.60	Nomeay	Complete
CUS128	22503	9/21/2823	Regeon	South	$15.80	LeadSource	Status
CO5183	25504	9/25/2823	Product	USA	$25.80	Creator	Complete
CO5128	25585	9/23/2023	Ropon	South	$25.80	LeadSource	Status
CO5188	23506	9/15/2023	Product	USA	$15.80	Creator	Complete
CO5181	25087	9/18/2022	Rogon	South	$15.80	LeadSource	Status
CO5122	28908	9/18/2823	Product	USA	$15.60	Nomeey	Complete
CO9533	23989	9/15/2023	Japan	South	$15.80	LeadSource	Status
CO9522	25989	9/15/2823	Product	USA	$15.80	LeadSource	Status

Column Micro-Adjustments: rename()

The rename() function resolves offending column names by accepting a dictionary mapping old values to new ones.

```
reviews.rename(columns={'points': 'score'})
```



Dictionary Mapping Schematic

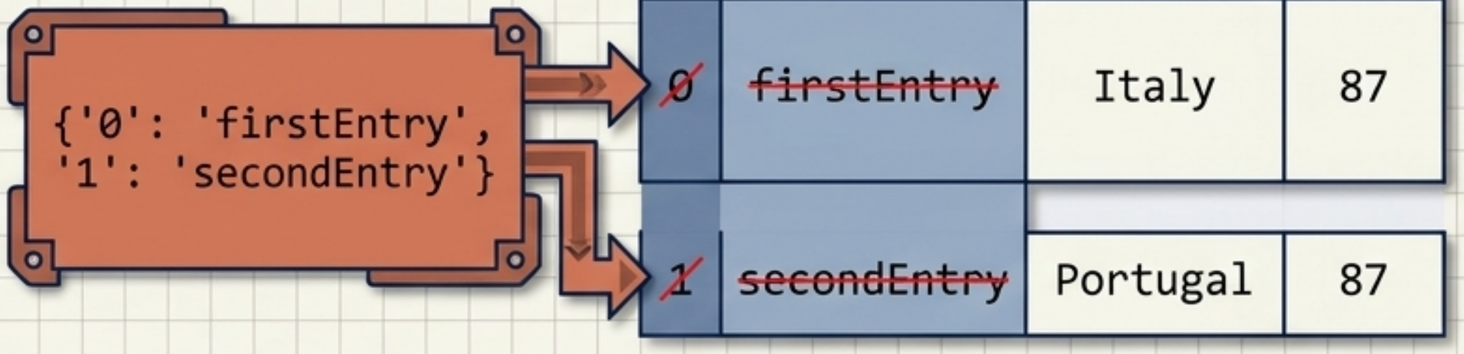
	country	designation	~~points~~	score
1	Italy	Vulkà Bianco	87	
2	Portugal	Avidagos	87	

Index Micro-Adjustments

`rename()` can also target index values using the `index` parameter, though renaming individual index values is rare.

```
reviews.rename(index={0: 'firstEntry', 1: 'secondEntry'})
```

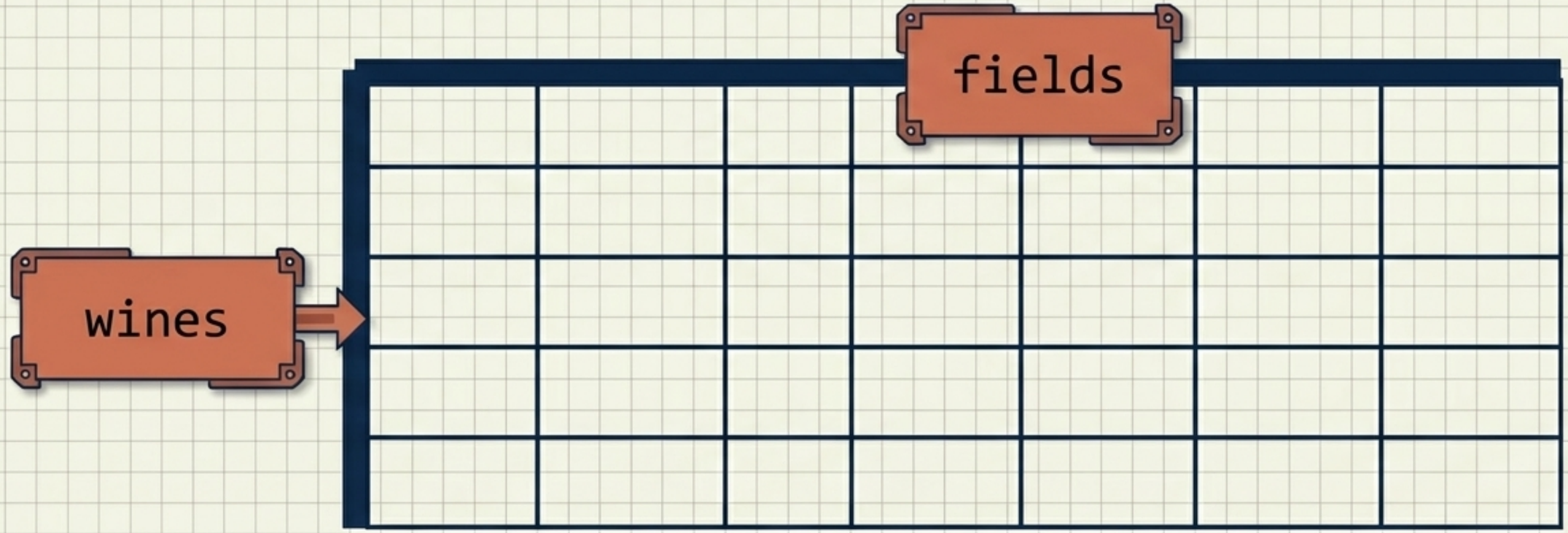
For comprehensive index replacement, `set_index()` is usually more convenient than mapping individual values via `rename()`.



Labelling the Blueprint: rename_axis()

Both the row index and the column index possess their own name attributes. `rename_axis()` assigns a master label to the entire axis.

```
reviews.rename_axis('wines', axis='rows').rename_axis('fields', axis='columns')
```

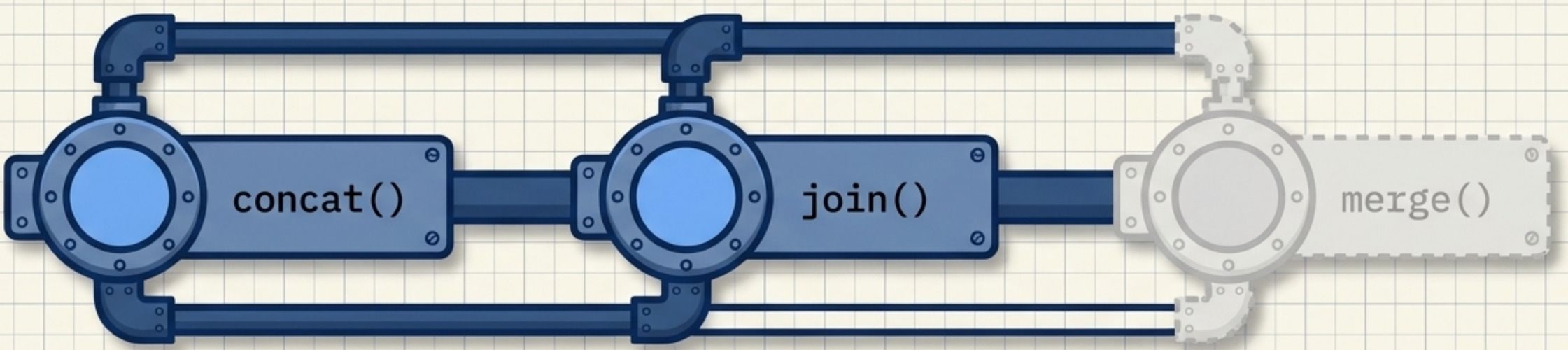


The Renaming Diagnostic Matrix

Function	Target	Primary Use
`rename()`	Specific Elements	Changing individual column headers or row names via dictionary mapping.
`set_index()`	Entire Index	Swapping out an entire existing index for a new one.
`rename_axis()`	Axis Attribute	Assigning a master title to the entire row or column structure.

Macro-Adjustments: Combining Data

Pandas offers three core methods for non-trivial combinations, scaling in complexity. We focus on the two most foundational: `concat()` and `join()`, as `join()` simplifies most of what `merge()` can achieve.

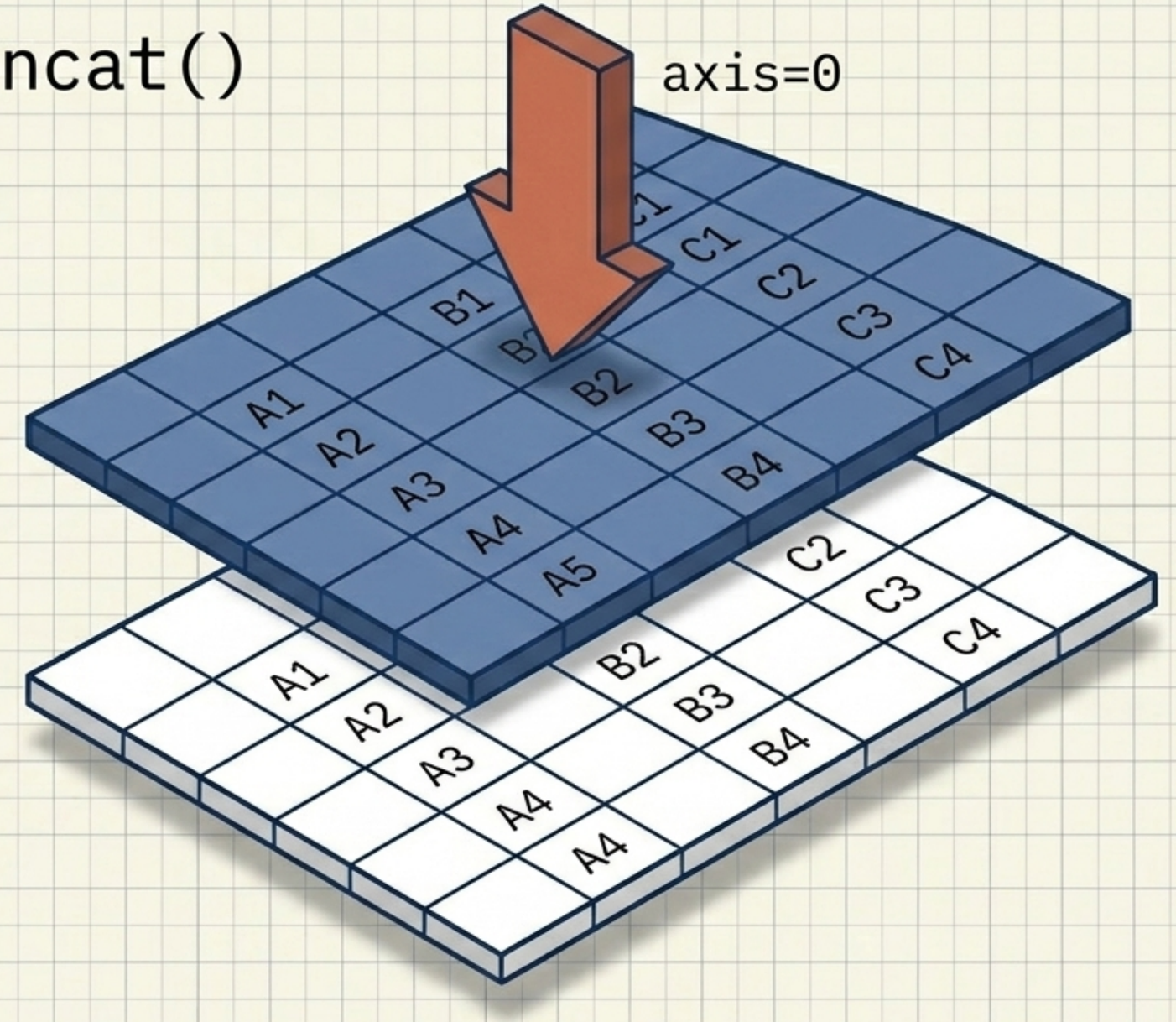


The Stacking Tool: `concat()`

Given a list of elements, `concat()` simply smushes them together along a defined axis.

It is the ideal tool when you have fragmented data with identical fields.

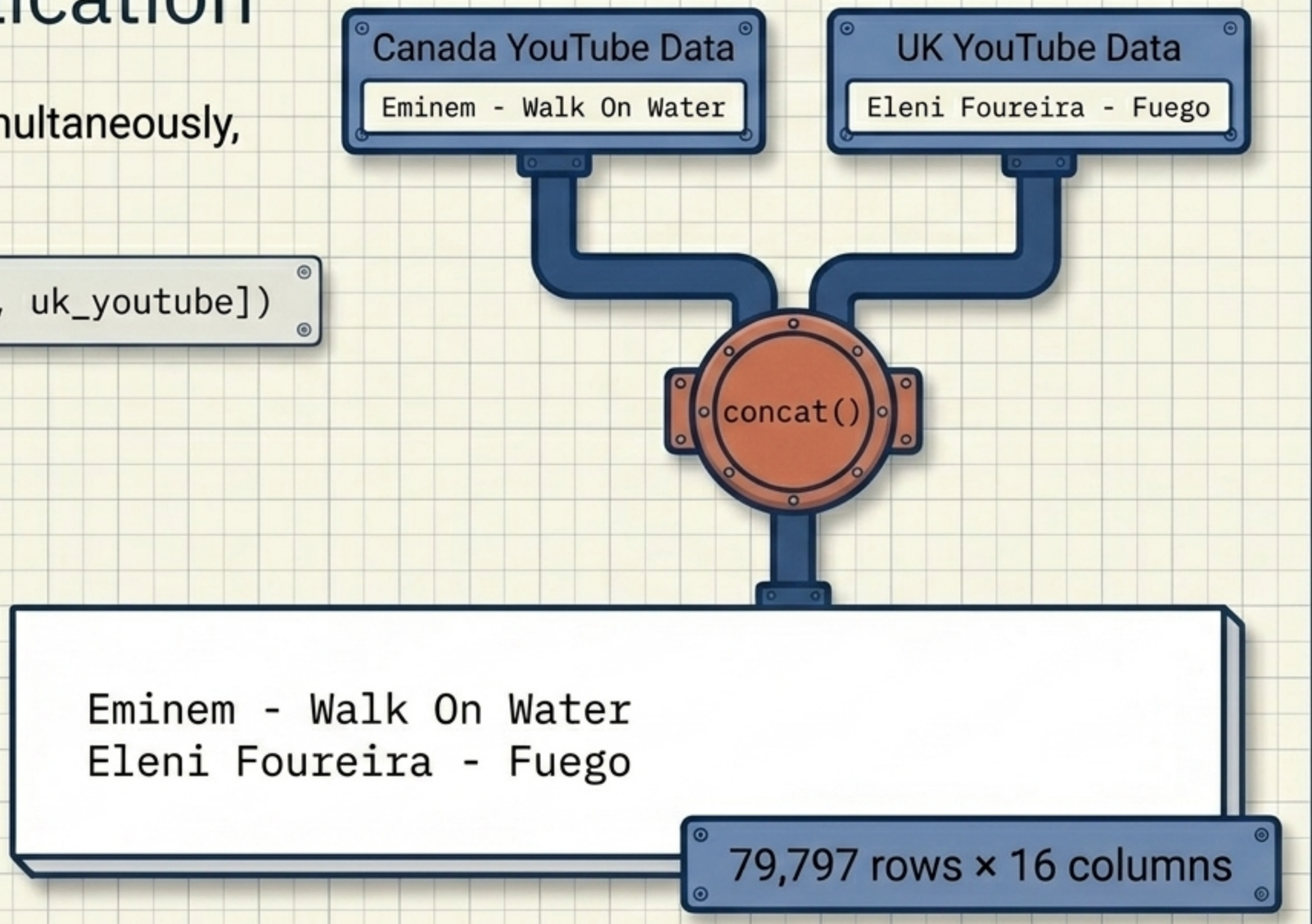
Mechanic Note: Assumes column schemas are identical.



concat() in Application

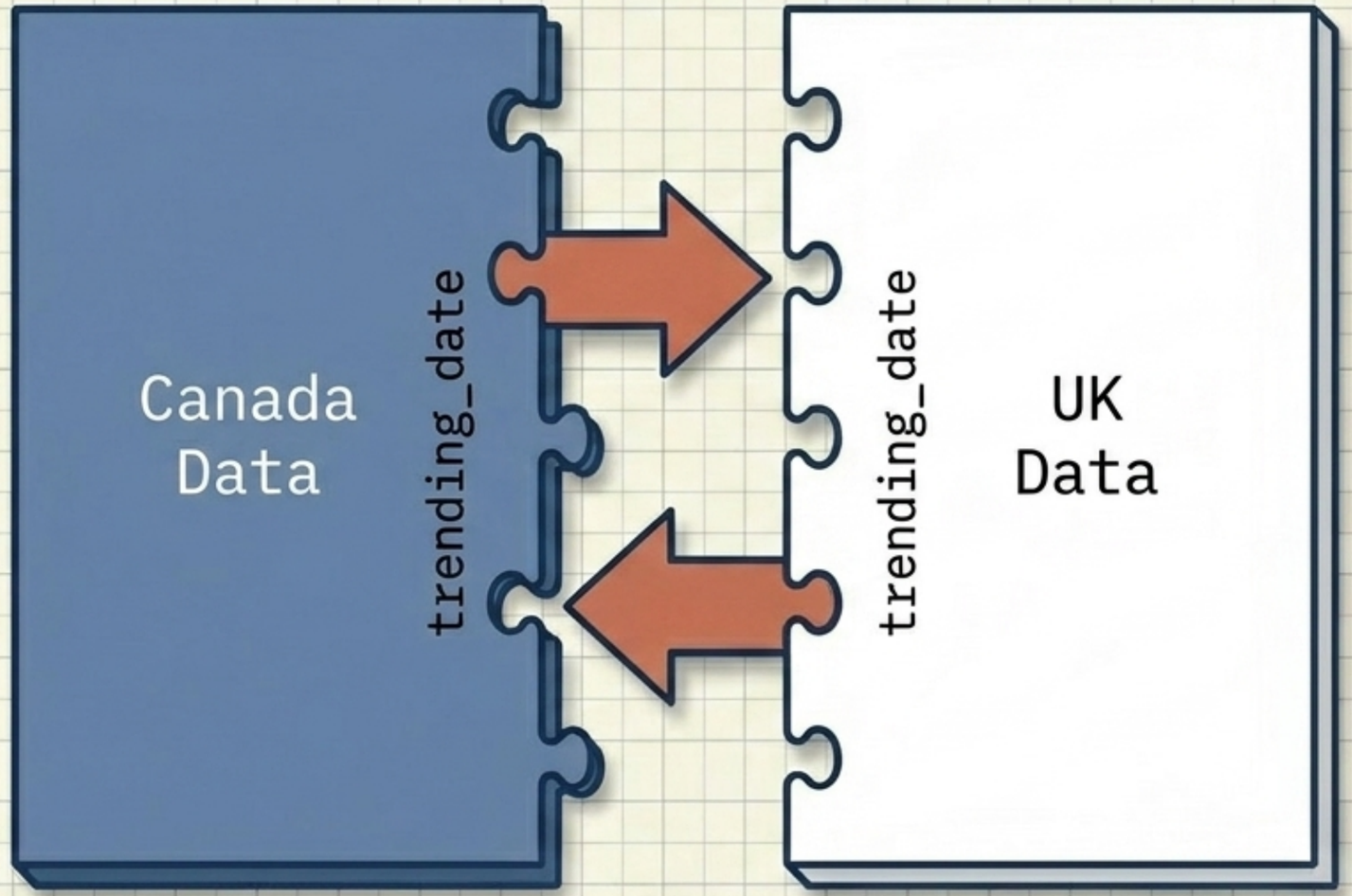
To study multiple countries simultaneously, stack their independent files.

```
pd.concat([canadian_youtube, uk_youtube])
```



The Interlocking Tool: `join()`

Combining datasets horizontally. `join()` links different DataFrames together based on a common index.

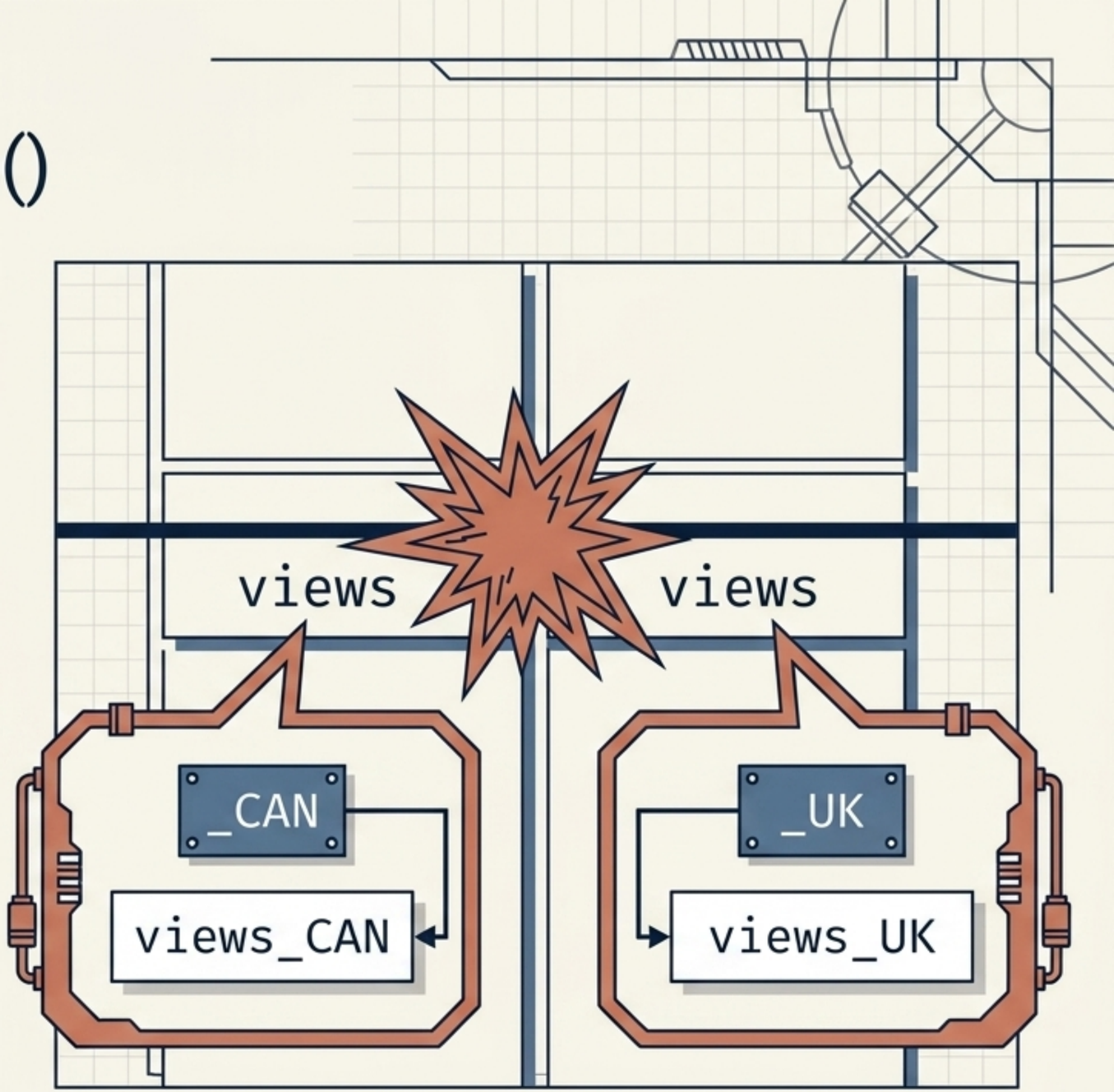


Use Case: Pulling down videos that were trending on the exact same day in both Canada and the UK.

Resolving Overlaps in join()

Because both datasets share identical column names, forcing them together creates a naming collision. The lsuffix and rsuffix parameters automatically append identifiers to resolve this.

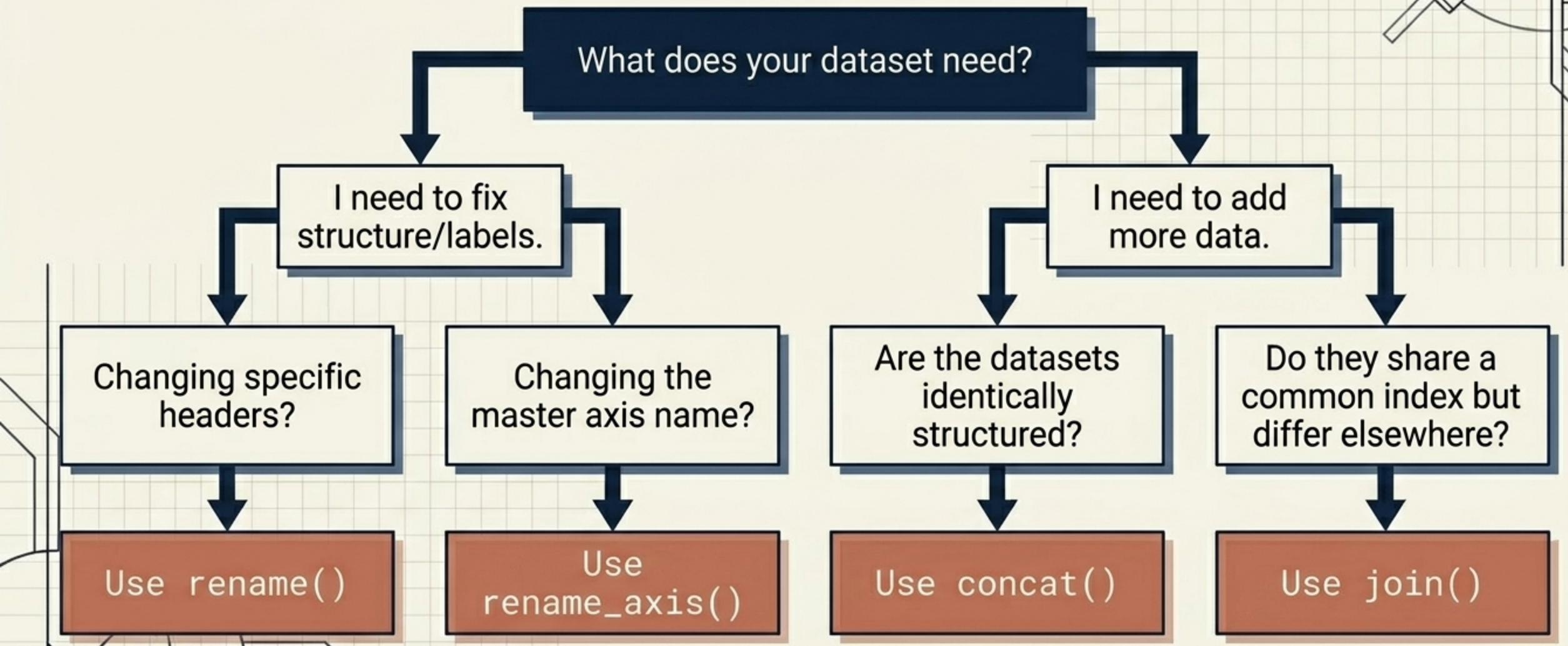
```
left.join(right, lsuffix='_CAN',
rsuffix='_UK')
```

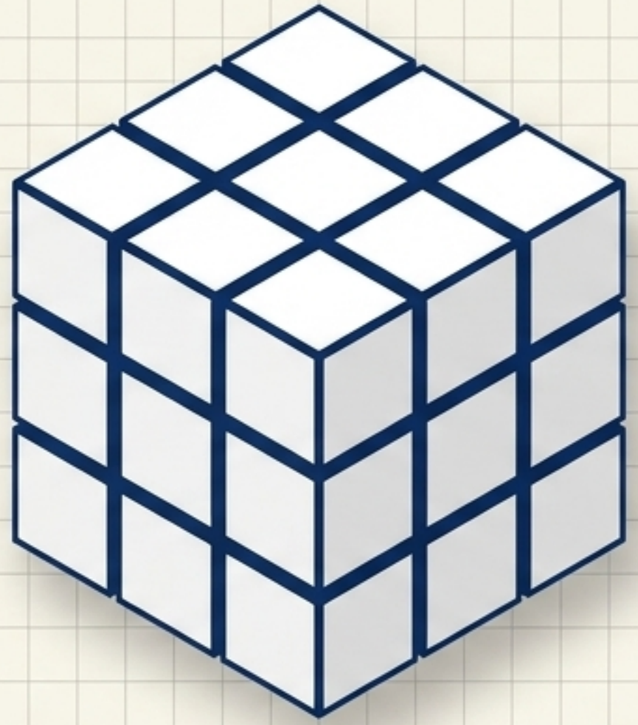


The Combining Diagnostic Matrix

Function	Mechanic	Primary Use Case
`concat()`	Vertical/Horizontal Stacking	Smushing structurally identical DataFrames together.
`join()`	Index Alignment	Merging different structural data side-by-side using a shared index.
`merge()`	Column/Index Alignment	Complex relational database-style joining (largely covered by join()).

The Data Architect’s Decision Tree





Data, Architected.

Keep this blueprint close. Mastering these fundamental structural tools transforms messy inputs into analytical clarity.